4. Project Design Phase

4.1 Introduction

The design phase transforms the requirements into a structured system design that defines the components, data flow, and functionality of PocketSmart AI. The system follows a modular architecture consisting of a user interface, an application server, a database, and a Gemini AI integration module that communicate through well-defined APIs.

4.2 System Design

PocketSmart AI follows a client-server architecture. The user interacts with the frontend, which sends requests to the FastAPI backend through REST APIs. The backend validates requests, communicates with the database, builds prompts that include the user's relevant financial data, calls the Google Gemini API, processes the response, and returns it to the frontend. The Gemini API key is kept only on the server.

4.3 System Architecture

- **Presentation Layer:** login, registration, dashboard, expense tracker, budget planner, insights, goals, recommendations, chat, and history pages.
- **Application Layer:** Python and FastAPI handling authentication, request validation, prompt construction, calculations, and business logic.
- **AI Layer:** Google Gemini model that generates categories, budget plans, insights, saving plans, recommendations, and chat answers.
- **Data Layer:** SQLite or PostgreSQL storing users, transactions, budgets, goals, recommendations, and chat history.

4.4 System Architecture Flow

User → Frontend → FastAPI Backend → Prompt Builder → Gemini API → Backend → Database → Frontend → User

4.5 Module Design

Module	Design Description
Authentication	Registration, login with password hashing, JWT token issuing and verification, profile management, logout.
Expense Tracking	Create, read, update, and delete transactions; optional Gemini category suggestion; filtering by date and category.
AI Budget Planner	Takes income, fixed costs, and goals; requests a category-wise budget (JSON) from Gemini; compares it with actual spending.
Spending Insights	Aggregates recent transactions, sends a summary to Gemini, and returns insights and saving opportunities.
Savings Goals	Takes goal, target amount, and date; calculates required monthly saving; Gemini adds practical tips; tracks progress.
Smart Recommendations	Takes need and budget limit; returns ranked alternatives with reasons and estimated cost range.

Module	Design Description
AI Finance Chat	Receives question and recent context; returns and stores the answer.
History & Dashboard	Retrieves saved items and aggregates income, expenses, categories, and savings for charts.

4.6 Database Design

Table	Main Fields
users	id, name, email, password_hash, currency, monthly_income, created_at
transactions	id, user_id, type (income/expense), category, amount, description, txn_date, created_at
budgets	id, user_id, month, total_income, plan_json, created_at
goals	id, user_id, name, target_amount, saved_amount, target_date, plan_json, created_at
recommendations	id, user_id, need, budget_limit, result_json, created_at
chat_history	id, user_id, question, answer, created_at

4.7 API Design

Method	Endpoint	Purpose
POST	/auth/register	Create a new user account
POST	/auth/login	Authenticate and return JWT
POST	/transactions	Add an income or expense entry
GET	/transactions	List transactions with filters
PUT / DELETE	/transactions/{id}	Edit or delete an entry
POST	/budget/generate	Generate an AI budget plan
GET	/insights	Generate spending insights
POST	/goals	Create a savings goal and plan
POST	/recommend	Get purchase recommendations within a budget
POST	/chat	Ask the finance assistant a question
GET	/history	Retrieve the user's saved items
GET	/dashboard	Retrieve financial statistics

4.8 Security Design

- Passwords are stored as hashes, never as plain text.
- JWT tokens protect all private endpoints, and each query is filtered by user_id.
- The Gemini API key is stored in environment variables on the server.
- User input is validated and sanitized before being sent to the AI model.

- Only the minimum necessary financial data is sent to the AI model; no bank credentials are collected.